pst2pdf

Converts an Outlook .pst archive into a small number of PDF review volumes built for redaction in Adobe Acrobat, Bates-numbered unless `--no-bates` is given. Every message becomes pages in a volume, with its converted attachments directly behind it, and the output carries an index, a START HERE guide, and a word list Acrobat's Search & Redact can import. Before a full run, `--survey` counts the pages and `--triage` lets the reviewers decide which attachments to produce at all. Runs in a Docker container with no network access.

It is built for responding to a records or discovery request: someone exports the search results from Microsoft 365 (or any mail system that writes .pst files), pst2pdf converts the export, and reviewers who are not necessarily technical redact the result in Acrobat. Every output decision is judged by whether it shortens that redaction pass, and whether an unredacted copy of a name could survive anywhere outside visible page content.

Why review volumes

A tree of one PDF per message, with attachments left as Word, Excel and image files, costs the reviewer most of their time in file handling: Acrobat's Search & Redact can search a folder of PDFs, but Apply and Save are per file, and native attachments cannot be redacted until someone converts them. So pst2pdf produces a few large volumes instead:

- **One pass per volume.** Every attachment is converted to pages directly behind its message, so one Search & Redact covers messages and attachments alike. A volume closes at a message boundary before it reaches the page limit (1,000 by default), because Acrobat slows down on very large files.
- **Repeated content once.** Reply chains quote the whole thread and the same attachment rides along on every reply. Messages are grouped by thread so related content lands on adjacent pages, duplicate messages appear once, and an identical attachment is included once with a cross-reference.
- **Acrobat's own tools.** Search & Redact takes an imported word list, so the run writes one from every sender and recipient. Sanitize strips bookmarks, links, metadata and embedded files, so everything a recipient needs is on the page.
- **No hidden channels.** A PDF can carry unredacted data outside the visible page: link targets, bookmark titles, metadata, image alt text, embedded files, and text laid out past the edge of the page, which is cut from view but stays in the text layer. Redaction only reaches what the reviewer can see, so the converter creates none of these.
- **Citable pages.** Productions are normally Bates-numbered, so every page has an ID for redaction logs and completeness questions. `--no-bates` turns that off for a small production, where a high page number invites the question of what else was held back.
- **Untrusted files.** Converting attachments means running LibreOffice on whatever was sent. The container runs with no network, per-file timeouts and size limits.

Before you start: Docker

pst2pdf is not installed the way an ordinary program is. It runs inside a Docker container: a sealed, minimal Linux system that holds the tool and everything it depends on. The container starts for one conversion and is thrown away when it ends. Docker is the program that builds and runs containers. You install it once, and the `pst2pdf.sh` script does the rest, so you never need to type a `docker` command yourself.

Why a container

- **Dependencies.** A conversion needs readpst to open the .pst, LibreOffice to convert Word, Excel and PowerPoint attachments, WeasyPrint and its Pango and Cairo libraries to lay out the pages, and a particular set of fonts, including Chinese, Japanese, Korean and emoji. Installing all of that at matching versions on every operating system would be a project of its own. The container carries all of it.
- **The same output everywhere.** Every run uses the same software and the same fonts, so an archive produces the same pages on any computer, and a page count from `--survey` holds for the full run.
- **Isolation.** An archive is untrusted input. Its attachments came from anyone who sent mail, and converting them means opening them in LibreOffice. The container runs with no network access and no special privileges, and it sees only the files it needs (below). Your data never leaves your computer, and remote images in messages are never fetched. PRIVACY.md has the details.

Installing Docker

Install Docker Desktop, or any other Docker-compatible runtime, and start it. To check that it is ready, open a terminal and run:

```
docker info
```

If that prints details about the server rather than an error, you are set.

- **macOS.** Install and start the runtime. The script finds the `docker` command on its own.
- **Linux.** Docker Engine is enough, without Desktop. Add your account to the `docker` group. Otherwise the script cannot reach Docker without `sudo`, and it reports that Docker is not running.
- **Windows.** `pst2pdf.sh` is a bash script, so run it from a WSL (Windows Subsystem for Linux) terminal with Docker Desktop's WSL integration turned on.

The script was developed on macOS. It has not been tested on Linux or WSL.

The first run

The first time you run the script, it builds the pst2pdf image, the template every container starts from. That takes a few minutes, needs an internet connection to download the software, and uses about 1.2 GB of disk space. Later runs reuse the image and start right away.

After you update pst2pdf, with `git pull` or by downloading it again, the first run rebuilds the image the same way, so it also takes a few extra minutes and needs an internet connection. The script notices the update by itself.

What the container can see

Only what the script hands it:

- the archive, read-only
- the output directory, the only place it can write
- a `--terms`, `--decisions` or `--search-report` file, read-only, when you give one (from a report folder, only its Summary CSV)

Nothing else on your computer is visible inside it, not even other files in the archive's folder. It runs as your user account, so the files it writes belong to you.

Usage

Download or clone this repository and open a terminal in its folder. The `pst2pdf.sh` script handles Docker for you. It finds the docker CLI, builds the image on first use, shows live progress, and prints a summary when done:

```
./pst2pdf.sh archive.pst "Request 2026-0001"
```

The request name appears on the START HERE page and in the PDF titles. When it is left out, the tool uses "Request" and today's date, never the archive's file name, because a file name can be a person's name and the PDF title survives redaction. Output lands in an `archive-pdfs` directory beside the input file. Options:

```
-o, --output DIR        Output directory
-f, --force             Allow converting into a non-empty output directory
-j, --jobs N            Work on N messages in parallel (default: one per CPU core)
-v, --verbose           Show every log line instead of the progress counter
    --rebuild           Rebuild the Docker image even if it is current (the script
                        rebuilds it by itself after an update)
    --zip               Also write a zip of the output directory beside it (never with
                        the unredacted original attachments, except for a --triage run)
    --search-report PATH  Show the Purview search on START HERE, exactly as Purview
                        recorded it: the export's report (zip, folder or Summary CSV),
                        or a text file holding only the query (see below)
```

```
-h, --help              Show the options and exit

Layout:
    --order MODE        thread (default), chrono, or folder
    --no-bates          Leave the pages unnumbered, with no page references in the
                        index, the exceptions table, or the notes
    --bates-prefix P    Page number prefix (default: DOC-)
    --bates-start N     First page number (default: 1)
    --volume-pages N    Pages per volume before a new one starts (default: 1000)
    --appendix-over N   Move converted attachments longer than N pages into separate
                        attachment volumes (0 moves them all; see below)
    --triage            Decide which attachments to produce first (see below)
    --survey            Count pages only, with no volumes (see below)

Content:
    --no-dedup-messages       Keep every copy of a duplicated message
    --no-dedup-attachments    Repeat identical attachments in full
    --no-convert-attachments  A page naming each attachment instead of converting it
    --keep-natives            Also write the original attachment files to a folder
    --no-expand-zips          Leave zip attachments unexpanded (their contents are listed)
    --decisions FILE          The reviewers' decisions: a --triage run's attachment index
                              or a --survey run's CSV, with the decision column filled in
    --withheld-by NAME        Who the notes say withheld the earlier copies of a report
                              (default: "withheld at the reviewer's direction")

Pre-marked redactions (optional):
    --terms FILE        Pre-mark every occurrence of the names/terms in FILE for redaction
    --patterns LIST     Also pre-mark patterns: phone, ssn, dob, student-id
    --student-id-regex RE     Regular expression for the student-id pattern
```

Examples

The paths and request names below are placeholders. Put options before the archive, and the request name, in quotes, after it. Each run needs an output directory of its own: the script refuses to write into one that already has files in it, so that two runs are never mixed. Delete the old directory or pick a new name with `-o`. (`--force` overrides the check, but files from the earlier run that the new one does not replace stay behind.)

A typical request. Volumes go to `export-pdfs`, beside the archive. Open `00 START HERE.pdf` first.

```
./pst2pdf.sh ~/Requests/0001/export.pst "Request 2026-0001"
```

Choosing the output directory, and zipping it. `--zip` also writes `volumes.zip` beside the directory, ready to hand to the reviewers.

```
./pst2pdf.sh -o ~/Requests/0001/volumes --zip ~/Requests/0001/export.pst "Request 2026-0001"
```

Recording the Purview search. Download the export's report from Purview's Process manager along with the export itself, and pass it with `--search-report`. START HERE then shows the query the export ran. The zip or the folder it unzips to both work.

```
./pst2pdf.sh --search-report ~/Requests/0001/Reports-0001.zip ~/Requests/0001/export.pst "Request 2026-0001"
```

A small production without page numbers. A six-digit Bates number on a production of a few dozen messages can suggest that far more was withheld. `--no-bates` leaves the pages unnumbered, and the index refers to messages by number instead.

```
./pst2pdf.sh --no-bates export.pst "Request 2026-0001"
```

Your own Bates numbering. Pages are numbered `CITY-001001`, `CITY-001002`, and so on.

```
./pst2pdf.sh --bates-prefix CITY- --bates-start 1001 export.pst "Request 2026-0001"
```

Pre-marking names for redaction. Write the names in a text file, one per line, with a category in brackets:

```
[student]  Jane Doe +variants
[parent]   Robert Doe
```

Then pass it with `--terms`. `--patterns` also marks phone numbers, Social Security numbers and dates of birth. The marks are proposals that the reviewer applies or clears in Acrobat.

```
./pst2pdf.sh --terms names.txt --patterns phone,ssn,dob export.pst "Request 2026-0001"
```

A very large archive: count the pages first. `--survey` makes no volumes. It writes `Attachment survey.pdf`, which shows how many pages a full run would make and which attachments fill them.

```
./pst2pdf.sh --survey -o survey export.pst "Request 2026-0001"
```

If a few long attachments dominate, move them into separate volumes behind the messages, here those over 50 pages:

```
./pst2pdf.sh --appendix-over 50 -o volumes export.pst "Request 2026-0001"
```

Or, when the same report arrives every day, type `latest` in the `decision` column of `Attachment survey.csv` for that report, save it, and pass it back so only the most recent copy is produced:

```
./pst2pdf.sh --decisions "survey/Attachment survey.csv" -o volumes export.pst "Request 2026-0001"
```

Letting the reviewers choose attachments first. `--triage` converts no attachments. It writes each one once as the original file, with an index for the reviewers to mark up. Nothing in it is for release, and the zip's name says so.

```
./pst2pdf.sh --triage --zip -o triage export.pst "Request 2026-0001"
```

When the reviewers send back `Attachment index - DO NOT RELEASE.csv` with their decisions filled in, the production run converts only what they chose to produce:

```
./pst2pdf.sh --decisions "Attachment index - DO NOT RELEASE.csv" -o volumes export.pst "Request 2026-0001"
```

Seeing everything. `-v` shows every log line instead of the progress counter, which helps when a run fails or stalls.

Using Docker directly

To run the container without the script:

```
docker build -t pst2pdf .
docker run --rm --network none -v "$(pwd)":/data pst2pdf --request-id "Request 2026-0001" /data/archive.pst /data/output
```

The script cannot tell which version an image built this way holds, so it rebuilds it once the next time it runs.

Called this way, the tool takes the same options, plus a few the wrapper does not pass on: `--request-id NAME` (the wrapper's second argument), `--overlay CATEGORY=TEXT` (text printed over pre-marked redactions of a category, such as a statute citation, repeatable), `--work-dir DIR` (keep the intermediate files), and `--version`. `docker run … pst2pdf --help` lists them all.

Output layout

```
output/
├── 00 START HERE.pdf              what is inside, the Purview search, counts, the Acrobat steps, what may be released
├── Volume 01 of 04.pdf            index pages, then the exceptions table, then messages with their attachments
├── Volume 02 of 04.pdf
├── ...
├── Attachments 01 of 02.pdf       only with --appendix-over: long attachments, each behind a header page
```

```
├── Redaction word list.txt        every sender and recipient name and address (keep back)
├── Pre-marked terms - DO NOT RELEASE.txt   per-term hit counts, only with --terms or --patterns (keep back)
├── manifest.json                  counts, per-volume SHA-256, page ranges, options used (keep back)
└── Original attachments - DO NOT RELEASE/  only with --keep-natives: unredacted, never zipped
```

- **Volumes.** Messages are grouped by conversation thread (or by date or by the original folder with `--order`), and a volume closes when the next message would push it past the page limit. By default every page carries a Bates stamp such as `DOC-000412` in the footer, and the PDF page labels match, so Acrobat's page box shows the same number. With `--no-bates` the pages are unnumbered, and the index and the exceptions table refer to messages by number instead of page. All times are shown in the container's local zone with its name, such as `Tue, 16 Sep 2025 10:17 CDT`.
- **Each message** is its header block, then the body, then the pages of each attachment, with no cover sheet in between. The message header lists the attachments by name. It gives no message number, folder, or count of copies, since those would tell a requester how much was collected, and the notes that stand in for attachments name a message by its date and subject instead. The index, the exceptions table, and Acrobat's bookmarks carry the message numbers, and a triage copy, which is never released, keeps all three in the header. Word, Excel, PowerPoint, and OpenDocument files are converted with LibreOffice. Images, text, and HTML go through the same renderer as messages. Tab- and comma-separated exports are laid out as tables. PDFs are re-saved with scripts, form fields, annotations, and embedded files removed. Forwarded messages and calendar items are rendered like messages. A zip attachment gets one page listing its contents, and the files inside it follow as attachments of their own.
- **Attachments without pages of their own** get a one-page note in their place. Anything that cannot be converted gets a placeholder saying why and a line in the exceptions table behind the index in Volume 01, so its file name goes through the same redaction pass as everything else. A message whose layout cannot be rendered is shown as plain text under its header, or as the header alone, with a notice. It never disappears from the volumes.
- **Everything wraps to the page.** Mail is laid out for a screen, and a wide table or a long link would run past the right margin, where the text is cut from view but can still be searched and copied. Message bodies and HTML attachments are reflowed to fit: fixed widths and no-wrap styles are dropped, and a long unbroken run of characters (a link target) may break anywhere, while names and ordinary words break only at spaces. Every page pst2pdf lays out is then checked, and a message that still would not fit is shown as plain text with a notice saying why. START HERE says how many. PDF attachments and Office documents keep their own layout and are not checked.
- **Duplicates.** A message found in more than one folder appears once. An attachment identical to an earlier one is replaced by a one-page note pointing to the first copy's pages.
- **Bookmarks and metadata carry no content.** Bookmarks are message numbers and dates only. Links in bodies are rendered as visible text, never as link annotations. Image alt text is dropped. Nothing is embedded. The only copy of any name is on a visible page, where redaction reaches it.
- **Pre-marked redactions** (optional). With `--terms`, every occurrence of each listed name or phrase is marked for redaction in the volumes as a proposal only: the marks appear in

Acrobat's Redact tool as marked but not applied, and the reviewer decides. The term file is one term per line with an optional category, `[student] Jane Doe`, and `+variants` to add "Doe, Jane", "J. Doe", and "Doe". A category may not appear inside its term, since the category is printed over the redaction. `--patterns` marks phone numbers, Social Security numbers, and dates of birth the same way, and student IDs when `--student-id-regex` says what one looks like. START HERE and the manifest report only per-category counts. The terms themselves, with hit counts, go to `Pre-marked terms - DO NOT RELEASE.txt`. See `docs/PREMARK-VERIFICATION.md` for the Acrobat check to run once before relying on it.

The Purview search (`--search-report`)

Purview writes a report for every export, downloaded from the export's Process manager as `Reports-<case>-<search>-…zip`. Its `Summary-….csv` records the query the export ran, as Purview built it from the condition builder. `--search-report` reads that query, with the case, search and export names and when the export ran, and START HERE shows it in a section of its own, exactly as recorded, so nobody retypes or pastes it. Pass the zip, the folder Safari unzips it into, or the Summary CSV. From a folder the script mounts only the Summary CSV, not the per-item report beside it. The triage START HERE and the attachment survey show the same section, and the manifest records the query and the SHA-256 of the Summary it came from.

Purview writes `(c:s)` between keywords entered on separate lines and `(c:c)` before the conditions (dates, senders and so on) that every item also had to meet. START HERE explains whichever of the two the query uses. If a Purview administrator's permissions filter limited the search, START HERE says so. A query exactly 255 characters long is flagged, since some Purview reports cut text off at that length. The submitter's address and the export's item counts are not read: one export is often split into several .pst files, so its total would not match any one archive. Give the same report to each archive's run.

START HERE also gives the export's run time, from the start and end Purview recorded, to the second and never rounded, for when the time spent on a request can be charged. It is the export process alone: an earlier Generate statistics or sample run, and the time anyone spent building the search, are not in it, and START HERE says so. The manifest records it as `run_time_seconds`. With no usable end time, no run time is shown.

When there is no report, a text file holding only the query works too, and START HERE says the query was supplied rather than read from Purview. A file with no query stops the run before extraction.

The query often names people, so it appears only on pages that are kept back: START HERE, the survey and the manifest. It is never printed on a volume or in a PDF title.

Long attachments (`--appendix-over`)

`--appendix-over N` moves every converted attachment longer than N pages out of the message volumes into `Attachments 01 of 02.pdf` volumes written after them, so the messages stay together and can be redacted first. `0` moves every converted attachment. Each slot that carried one gets a

one-page note saying where it went. Identical copies appear in the attachment volumes once, and each item opens with a header page naming the file and the messages that carried it. Neither page gives an item number or says how many messages carried the file. Page numbers run on from the last message page.

Counting pages first (`--survey`)

For an archive that might run to tens of thousands of pages, `--survey` does everything a full run does except write the volumes, so every count is real:

```
output/
├── Attachment survey.pdf      projected pages and volumes, pages by type, the largest 25 attachments,
│                              and what --appendix-over at 10 to 500 pages would do (keep back)
├── Attachment survey.csv      every distinct attachment, most pages first, with its SHA-256
└── manifest.json
```

`--terms`, `--patterns`, and `--keep-natives` are ignored with a note, so the survey takes the same command line as the full run.

Attachment decisions (`--decisions`)

`--decisions FILE` takes the survey's `Attachment survey.csv`, or a triage run's attachment index (below), with its `decision` column filled in in Excel. A row added by hand may give only a `filename`, which matches any case and may be a pattern with `*` for a report whose name changes daily.

- **Blank or `produce`**: the attachment is produced. A `produce` row with a SHA-256 also exempts those exact bytes from the rules below.
- **`latest`**: every attachment with the row's file name is a copy of one recurring report, and only the copy from the most recent message is produced. Every other copy gets a one-page note saying where the kept copy is, and a line in the exceptions table. The note says the copy was "withheld at the reviewer's direction", or, with `--withheld-by "the records custodian"`, "withheld at the direction of the records custodian".
- **`omit`**: the attachment is not produced. A row with a SHA-256 covers exactly those bytes, never another file that shares the name. The row needs a note, and the note is printed exactly as written on the page where the attachment was and in the exceptions table. The tool adds no wording of its own, since whether an attachment is out of scope or not public is the reviewers' call.

A decision the tool does not know, or an `omit` without a note, stops the run before it starts, naming the line. A row that matches nothing is warned about. The manifest records every rule and what it matched.

Deciding which attachments to produce (`--triage`)

When most attachments may not need producing at all, a triage run comes first. It converts nothing:

```
output/
├── 00 START HERE.pdf                        what this folder is for and what to fill in
├── Triage copy - Volume 01 of 04.pdf        every message, a one-page note naming the file where each
attachment was
├── Attachment index - DO NOT RELEASE.csv    one row per file: ID, name, type, size, the messages that
carried it
├── Original attachments - DO NOT RELEASE/   each attachment once, exactly as sent, as "A0042 name.ext"
└── manifest.json
```

Nothing in it is for release, and the triage volumes are unnumbered and not for redaction. The reviewers open the index in Excel, type `omit` and a reason in the note column for each attachment not to be produced (or `latest` for a recurring report), and send it back. The production run then takes it as `--decisions` : only what is produced is converted, and each omitted attachment gets a page with the reviewers' note, exactly as written, where it was attached, and a line in the exceptions table. `--triage` cannot be combined with `--survey` or `--appendix-over` , and ignores `--terms` and `--patterns` .

Privacy and security

pst2pdf sends nothing anywhere. PRIVACY.md says what it writes, how the no-network rule is enforced, and how to check it yourself. To report a vulnerability, see SECURITY.md.

Development

```
.venv/bin/python -m pytest -q
```

Unit tests run on the host without WeasyPrint, LibreOffice, or readpst and skip the few that need them. The Docker image is the integration environment. See CLAUDE.md for the details and `docs/DESIGN.md` for the module contract.

This README is also published as `docs/README.html` , a single file that works offline, and `docs/README.pdf` . Both are generated from README.md. After editing it, regenerate them and commit all three:

```
.venv/bin/python scripts/readme_docs.py
```

The PDF is rendered inside the pst2pdf image, so it must be built. The tests fail when either file is out of date.

License

Copyright 2026 Special School District No. 1, Minneapolis Public Schools.

Licensed under the Apache License, Version 2.0. See LICENSE for the full text and NOTICE for the copyright notice.

Generated from README.md in the pst2pdf repository.